

I'll write a simple pagination implementation without DTOs or repository pattern - just basic Entity Framework with direct database access.

Simple Pagination (No DTOs, No Repository Pattern)

1. Simple Pagination Parameters

```
public class PaginationParams
{
    public int PageNumber { get; set; } = 1;
    public int PageSize { get; set; } = 10;
    public string? Search { get; set; }
    public string? SortBy { get; set; }
    public bool SortDesc { get; set; } = false;
}
```

2. Simple Paged Result

```
public class PagedResult
{
    public List Data { get; set; } = new List();
    public int CurrentPage { get; set; }
    public int PageSize { get; set; }
    public int TotalCount { get; set; }
    public int TotalPages { get; set; }
    public bool HasNextPage => CurrentPage < TotalPages;
    public bool HasPreviousPage => CurrentPage > 1;
}
```

3. Simple Controller with Direct EF Pagination

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

[ApiController]
[Route("api/[controller]")]
public class EmployeesController : ControllerBase
{
    private readonly ApplicationDbContext _context;

    public EmployeesController(ApplicationDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<IActionResult> GetEmployees([FromQuery]
    PaginationParams params)
    {
        var query = _context.Employees.AsQueryable();

        // Apply search filter
        if (!string.IsNullOrEmpty(params.Search))
        {
            query = query.Where(e =>
                e.Name.Contains(params.Search) ||
                e.Email.Contains(params.Search));
        }

        // Apply sorting
        if (!string.IsNullOrEmpty(params.SortBy))
        {
            switch (params.SortBy.ToLower())
            {
                case "name":
                    query = params.SortDesc
                        ? query.OrderByDescending(e => e.Name)
                        : query.OrderBy(e => e.Name);
            }
        }
    }
}
```

```

        break;
    case "email":
        query = params.SortDesc
            ? query.OrderByDescending(e => e.Email)
            : query.OrderBy(e => e.Email);
        break;
    case "salary":
        query = params.SortDesc
            ? query.OrderByDescending(e => e.Salary)
            : query.OrderBy(e => e.Salary);
        break;
    default:
        query = query.OrderBy(e => e.Id);
        break;
    }
}
else
{
    query = query.OrderBy(e => e.Id);
}

// Get total count
var totalCount = await query.CountAsync();

// Apply pagination
var items = await query
    .Skip((params.PageNumber - 1) * params.PageSize)
    .Take(params.PageSize)
    .ToListAsync();

var result = new PagedResult<Employee>
{
    Data = items,
    CurrentPage = params.PageNumber,
    PageSize = params.PageSize,
    TotalCount = totalCount,
    TotalPages = (int)Math.Ceiling(totalCount /
(double)params.PageSize)
};

```

```

        return Ok(result);
    }

    // Regular CRUD operations
    [HttpGet("{id}")]
    public async Task<IActionResult> GetEmployee(int id)
    {
        var employee = await _context.Employees.FindAsync(id);
        if (employee == null)
            return NotFound();

        return Ok(employee);
    }

    [HttpPost]
    public async Task<IActionResult> CreateEmployee([FromBody]
Employee employee)
    {
        _context.Employees.Add(employee);
        await _context.SaveChangesAsync();
        return CreatedAtAction(nameof(GetEmployee), new { id =
employee.Id }, employee);
    }

    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateEmployee(int id, [FromBody]
Employee employee)
    {
        if (id != employee.Id)
            return BadRequest();

        _context.Entry(employee).State = EntityState.Modified;
        await _context.SaveChangesAsync();
        return NoContent();
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteEmployee(int id)

```

```

{
    var employee = await _context.Employees.FindAsync(id);
    if (employee == null)
        return NotFound();

    _context.Employees.Remove(employee);
    await _context.SaveChangesAsync();
    return NoContent();
}
}

```

4. Basic Employee Entity

```

using System.ComponentModel.DataAnnotations;

public class Employee
{
    public int Id { get; set; }

    [Required]
    public string Name { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    public decimal Salary { get; set; }

    public string Department { get; set; } = string.Empty;

    public DateTime CreatedDate { get; set; } = DateTime.Now;
}

```

5. Simple DbContext

```
using Microsoft.EntityFrameworkCore;

public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options) : base(options)
    {
    }

    public DbSet<Employee> Employees { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Employee>(entity =>
        {
            entity.HasKey(e => e.Id);
            entity.Property(e =>
e.Name).IsRequired().HasMaxLength(100);
            entity.Property(e =>
e.Email).IsRequired().HasMaxLength(100);
            entity.Property(e => e.Department).HasMaxLength(50);
            entity.HasIndex(e => e.Email).IsUnique();
        });
    }
}
```

6. Program.cs Setup

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("Defaul
```

```
tConnection"))));

// Add Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

7. Usage Examples

Basic pagination

GET /api/employees?pageNumber=1&pageSize=10

With search

GET /api/employees?pageNumber=1&pageSize=10&search=john

With sorting

GET /api/employees?pageNumber=1&pageSize=10&sortBy=name&sortDesc=true

Combined

GET

</api/employees?pageNumber=2&pageSize=5&search=@example.com&sortBy=salary>

8. Response Format

```
{
  "data": [
    {
      "id": 1,
      "name": "John Doe",
      "email": "john@example.com",
      "salary": 50000,
      "department": "IT",
      "createdDate": "2024-01-01T00:00:00"
    }
  ],
  "currentPage": 1,
  "pageSize": 10,
  "totalCount": 25,
  "totalPages": 3,
  "hasNextPage": true,
  "hasPreviousPage": false
}
```

Complete Simple Pagination Implementation

This is a **minimal, straightforward pagination implementation** without DTOs or repository pattern - just direct Entity Framework access.

Key Features:

- **Simple Classes:** Only 2 classes needed (PaginationParams, PagedResult)
- **Direct EF:** No repository pattern, direct database access
- **No DTOs:** Uses the entity directly
- **Search & Sort:** Built-in search and sorting functionality
- **Efficient:** Uses `Skip()` and `Take()` for database pagination

How It Works:

1. **Controller** receives query parameters (pageNumber, pageSize, search, sortBy)
2. **Builds query** with optional search and sorting
3. **Gets total count** for pagination metadata
4. **Applies pagination** using `Skip()` and `Take()`
5. **Returns structured response** with data + pagination info

Performance Benefits:

- Only loads requested records (not all data)
- Efficient SQL with OFFSET and FETCH
- Fast response times even with large datasets
- Reduced memory usage

Usage:

GET

`/api/employees?pageNumber=2&pageSize=10&search=john&sortBy=name&sortDesc=true`

This simple approach is perfect for small to medium applications where you don't need the complexity of full repository pattern or DTOs, but still want efficient pagination.